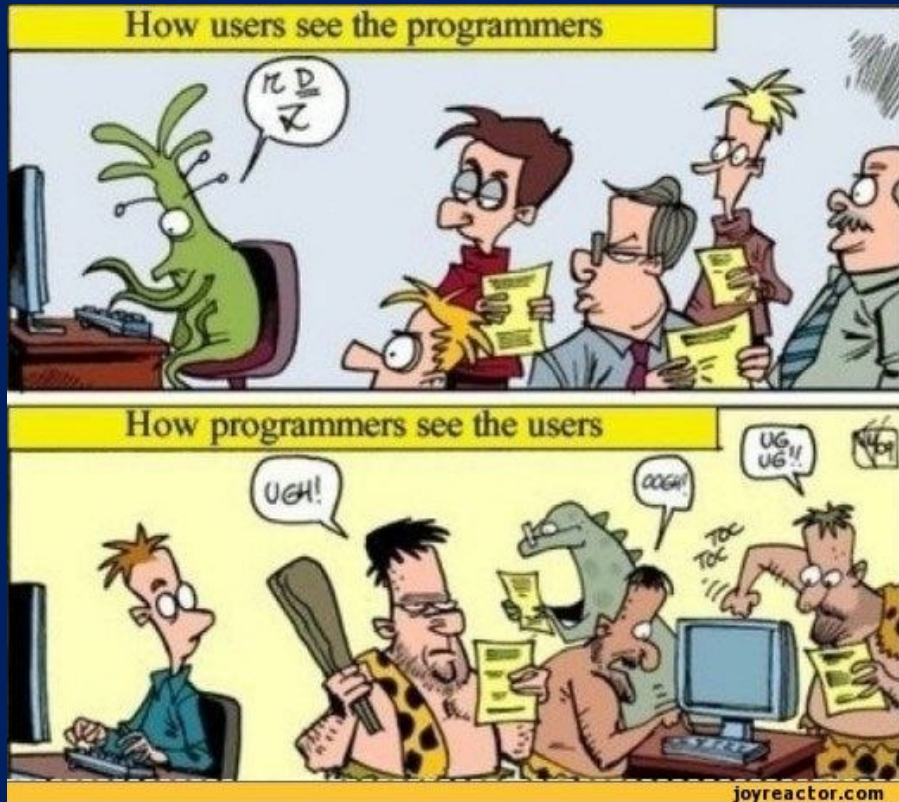


Week 5: How to work with Views

Chapter 7, 15

INFT3050 – Web Programming



WARNING

This material has been reproduced and communicated to you by or on behalf of the University of Newcastle in accordance with section 113P of the *Copyright Act 1968* (the Act). The material in this communication may be subject to copyright under the Act. Any further reproduction or communication of this material by you may be the subject of copyright protection under the Act.

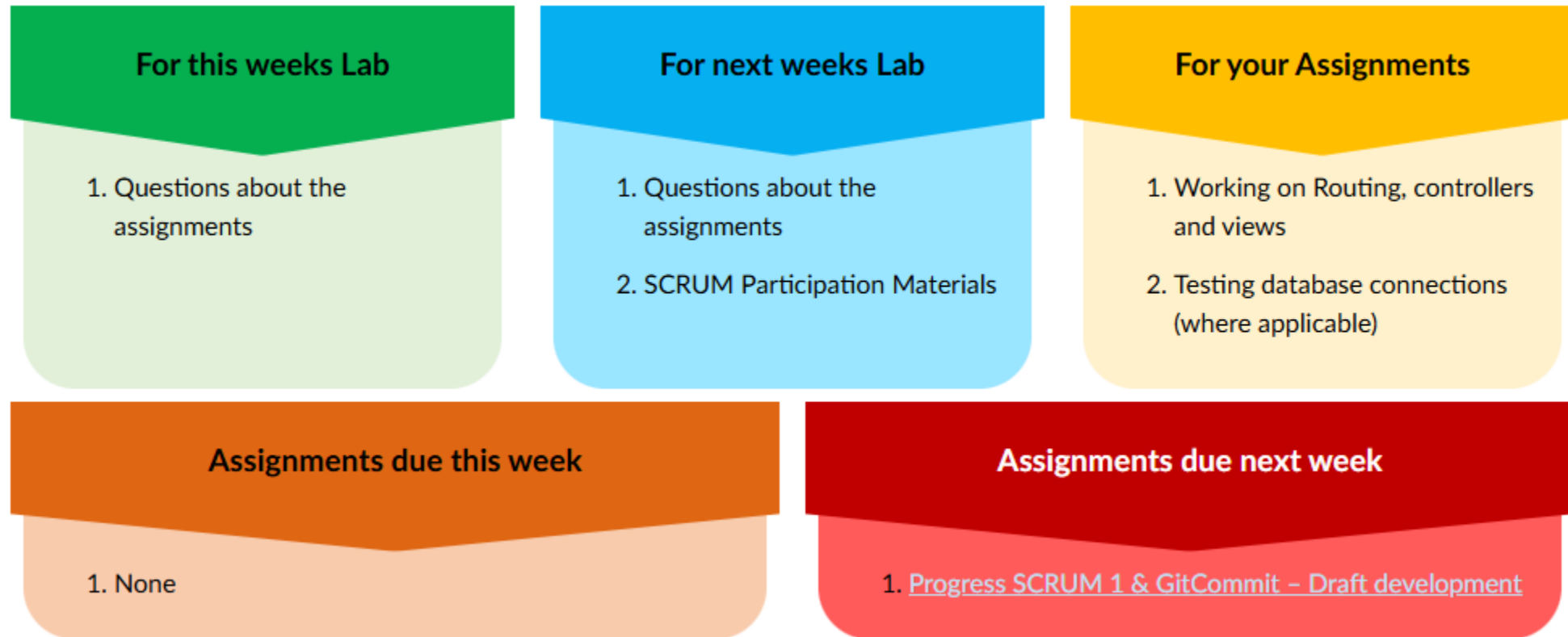
Do not remove this notice



Contents

- Housekeeping
- Working with Razor views
- Working with tag helpers, partial views, and view components
- Demo

Housekeeping



Housekeeping

Progress SCRUM 1 & GitCommit - Draft development

Assessment Type	Presentation and online submission
Purpose	In this assignment students present their progress for their project that will use the tools and techniques taught in the course.
Description	Maps to Course Learning Outcomes: 1, 2, 3, 4
Weighting	6%
Due Date	SCRUM
	Week 6, in-lab
	GitCommits and Individual Contributions
	Week 6, Sunday 23:59 AEST
Submission Method	In-Class and online
Assessment Configuration	This is an INDIVIDUAL assessment
Assessment Criteria	Demonstrated understanding of developing a prototype MVC Web App solution and the skills needed to develop them.
Return Method	Online
Feedback Provided	Online

Housekeeping

Deliverables

This assessment item takes the form of a brief 1-2 minute (per person) face-to-face meeting with your Senior Developers (Lecturer/Lab Demonstrators) in your Lab Session.

1) Participate in the SCRUM by bringing:

- Any questions you have about your assignment, team etc.
- Code you would like to discuss/show
- Evidence of individual development of your team's prototype solutions.
- Submit in the lab face-to-face

2) GitCommit

- This week you will submit code to your Github that demonstrates that the progress of your prototype development
- This must be new code that has been implemented - but not necessarily fully working please label this as 'in progress'
- The code must demonstrate an individual contribution from each team member
- Submit to GitHub Classroom [here](#) ➞

3) Submit evidence of your individual contributions to the project's development

- A brief 100-300 word explanation of your progress since the previous SCRUM
- This brief can include any screens of the team chat, code or relevant planning documents
- Submit to Canvas

Housekeeping

Deliverables

This assessment item takes the form of a brief 1-2 minute (per person) demonstration (by you and your team) in your Lab Session.

1) Participate in the SCRUM by bringing:

- Any questions you have about your assignment, team etc.
- Code you would like to discuss/show
- Evidence of individual development of your team's prototype solution
- Submit in the lab face-to-face

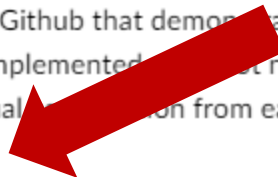
2) GitCommit

- This week you will submit code to your Github that demonstrates your team's progress
- This must be new code that has been implemented since the last SCRUM
- The code must demonstrate an individual contribution from each team member
- Submit to GitHub Classroom [here](#) 

3) Submit evidence of your individual contributions to the project's development

- A brief 100-300 word explanation of your progress since the previous SCRUM
- This brief can include any screens of the team chat, code or relevant planning documents
- Submit to Canvas

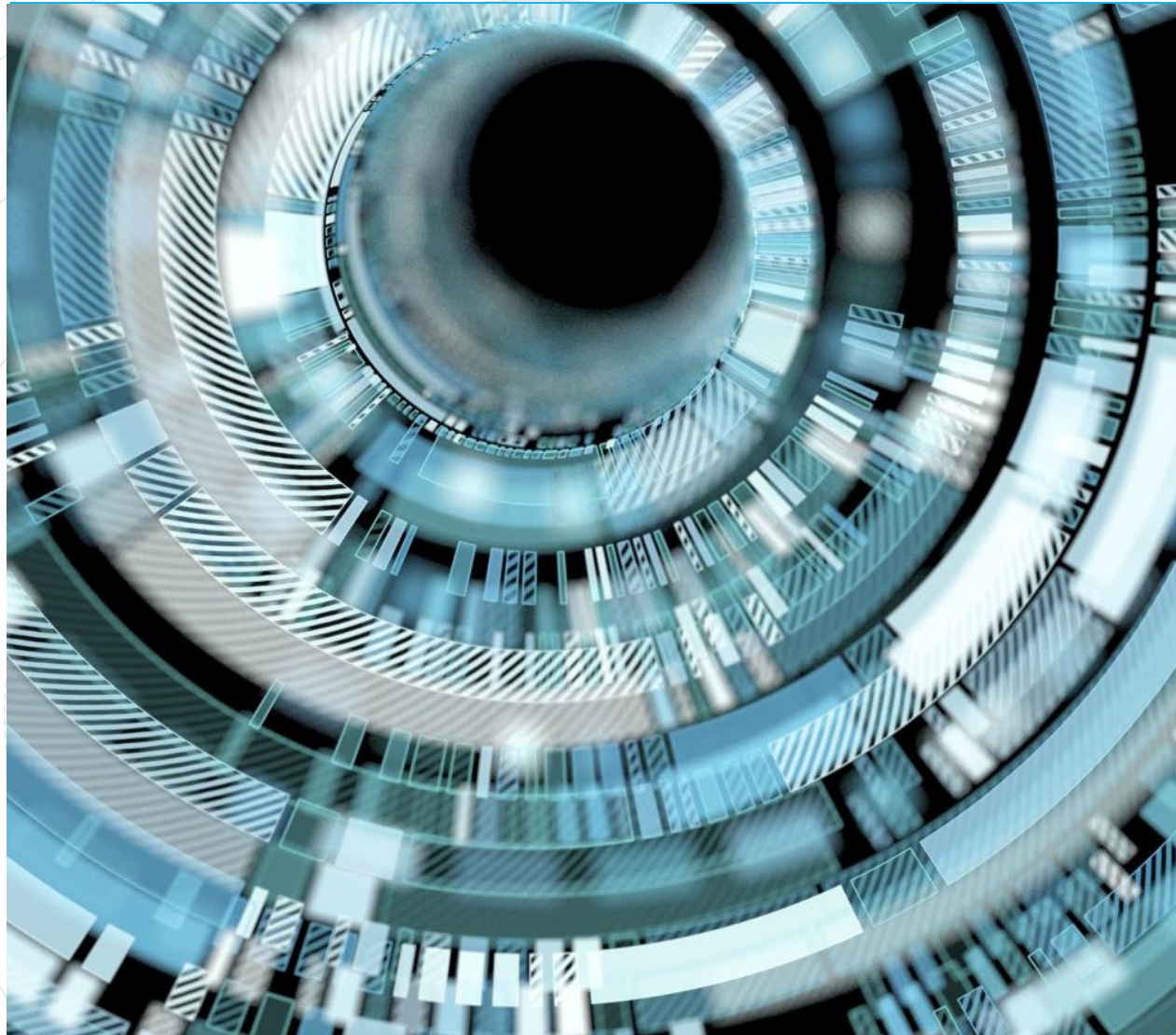
This is NOT the same as the first GitCommit
– please use the new link



Project Update

1. Keep working on Views
2. Review the Prototype documentation on Canvas
3. Work on database connections
4. Test connections and pulling data into your web-app
5. Test your models and controllers
6. Prep for SCRUM and GitCommit NEXT WEEK
7. Please ask questions to me or Connor to clarify anything about this project





How to work with Razor Views

Razor Syntax – Code Blocks and Inline Expressions

- To execute one or more C# statements, you can declare a Razor code block by coding the @ sign followed by a pair of braces ({ }). Within the braces, you can code one or more C# statements.
- To evaluate a C# expression and display its result, you can code an inline expression by coding the @ sign and then coding the expression within a pair of parentheses. For many expressions, the parentheses are optional.

The syntax for a Razor code block

```
@{  
    // one or more C# statements  
}
```

The syntax for an inline expression

```
@(csharp_expression)
```

The Index() action method of the Home controller

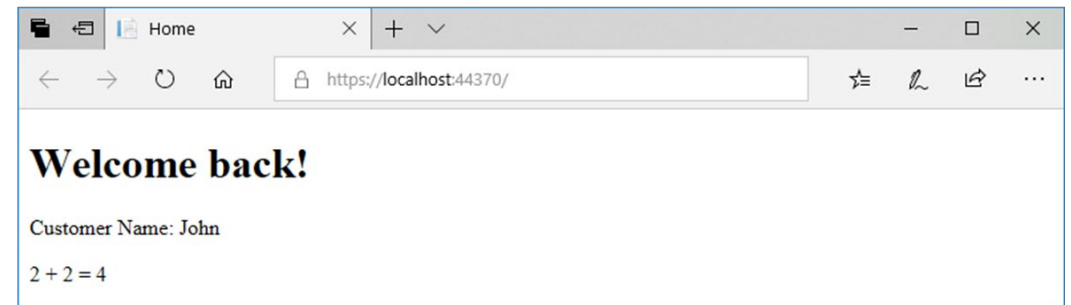
```
public IActionResult Index()  
{  
    ViewBag.CustomerName = "John";  
    return View();    // returns Views/Home/Index.cshtml  
}
```

Sample Code – Code Blocks and Inline Expressions

The Views/Home/Index.cshtml file

```
@{
    string message = "Welcome!";
    if (ViewBag.CustomerName != null)
    {
        message = "Welcome back!";
    }
}
<!DOCTYPE html>
<html>
<head>
    <meta name="viewport" content="width=device-width" />
    <title>Home</title>
</head>
<body>
    <h1>@message</h1>
    <p>Customer Name: @ViewBag.CustomerName</p>
    <p>2 + 2 = @(2 + 2)</p>
</body>
</html>
```

The view displayed in a browser



Coding inline loops

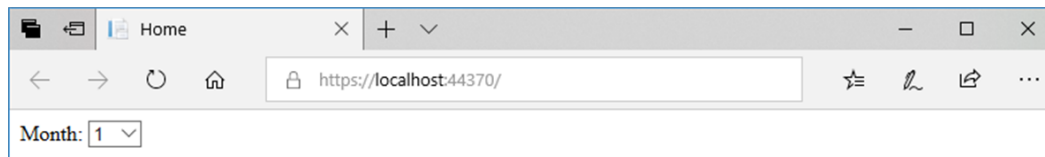
- You can code inline loops within a view. Within these loops, you can use HTML tags to send HTML to the view
- You can use Bootstrap CSS classes or tag helpers (see later on this lecture) – but the following examples do not – this will come later

Coding inline loops - Examples

A for loop that displays a drop-down list of month numbers

```
<label for="month">Month:</label>
<select name="month" id="month">
  @for (int month = 1; month <= 12; month++)
  {
    <option value="@month">@month</option>
  }
</select>
```

The result displayed in a browser



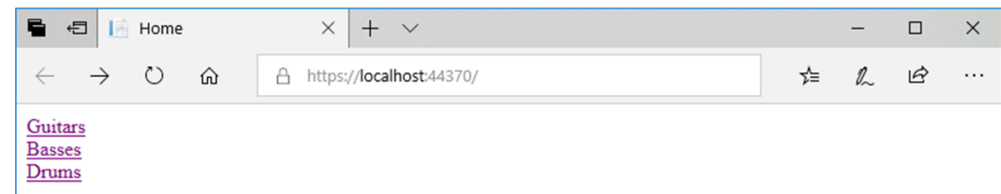
Code in a controller that creates a list of strings

```
public IActionResult Index()
{
    ViewBag.Categories = new List<string>
    {
        "Guitars", "Basses", "Drums"
    };
    return View();
}
```

A foreach loop that displays a list of links

```
@foreach (string category in ViewBag.Categories)
{
    <div>
        <a href="/Product/List/@category/">@category</a>
    </div>
}
```

The result displayed in a browser



Coding inline conditional statements

- You can code **inline conditional statements** such as if-else and switch statements within a view. These statements can use HTML tags to send HTML to the view.
- To send plain text to a view, you can use the `<text>` tag. This is useful for sending part of an HTML tag such as an HTML attribute or its value.
- You can code an **inline conditional expression** by using C#'s conditional operator (`? :`) to send HTML tags or plain text to the view. This works like other complex inline expressions that require parentheses

An if-else statement in a view

```
@if (ViewBag.ProductID == 1)
{
    <p>Fender Stratocaster</p>
}
else if (ViewBag.ProductID == 2)
{
    <p>Gibson Les Paul</p>
}
else
{
    <p>Product Not Found</p>
}
```

Coding inline conditional statements - Samples

A switch statement in a view

```
@switch (ViewBag.ProductID)
{
    case 1:
        <p>Fender Stratocaster</p>
        break;
    case 2:
        <p>Gibson Les Paul</p>
        break;
    default:
        <p>Product Not Found</p>
        break;
}
```

An if statement that adds a Bootstrap CSS class if true

```
<a asp-controller="Product" asp-action="List"
    asp-route-id="All"
    class="list-group-item"
    @if (ViewBag.SelectedCategoryName == "All") {
        <text>active</text>
    }">
    All
</a>
```

A conditional expression that adds a Bootstrap CSS class if true

```
<a asp-controller="Product" asp-action="List" asp-route-id="All"
    class="list-group-item"
    @ (ViewBag.SelectedCategoryName == "All" ? "active" : "")">
    All
</a>
```




Demo

Code Demos

- **Demo 1 - File:** Ch07aGuitarShop
- **Demo 2 – File:** Ch15Bookstore

The starting folders and files for an app

- By convention, you store the views for an MVC app in a series of folders and files whose names correspond to the controllers and action methods that return the views.
- The Program.cs file typically contains code that configures the middleware for the app including the routing that specifies how controllers and their action methods are mapped to URLs.
 - Routing was discussed last week

The starting folders and files for the Guitar Shop app

```
GuitarShop
  /Controllers
    /HomeController.cs
    /ProductController.cs
  /Models
    /Category.cs
    /Product.cs
  /Views
    /Home
      /Index.cshhtml -- the view for the Home/Index action
      /About.cshhtml -- the view for the Home/About action
    /Product
      /List.cshhtml -- the view for the Product/List action
      /Details.cshhtml -- the view for the Product/Details action
      /Update.cshhtml -- the view for the Product/Update action
    /Shared
      /_Layout.cshtml -- a layout that can be shared by views
      _ViewImports.cshtml -- imports models and tag helpers for views
      _ViewStart.cshtml -- specifies the default layout for views
  /wwwroot
    /css
      /custom.css
    /lib
      /bootstrap/css/bootstrap.min.css
  Startup.cs -- configures middleware that may impact views
  Program.cs -- sets up the app
```

Coding controllers that return Views

- A controller typically contains action methods that are mapped to the view files in the Views folder.
- We can use the `View()` method to create a `ViewResult` object that corresponds to the name of the current controller/action method
- You can also use the `View(name)` method to create a `ViewResult` object corresponding to the current controller and specify the name of the View

Coding controllers that return Views - Samples

The Home controller

```
public class HomeController : Controller
{
    public IActionResult Index()
    {
        return View();    // Views/Home/Index.cshtml
    }
}
```

The Product controller

```
public class ProductController : Controller
{
    public IActionResult Index()
    {
        return View("List");    // Views/Product/List.cshtml
    }

    public IActionResult List(string id = "All")
    {
        ViewBag.Category = id;
        return View();    // Views/Product/List.cshtml
    }

    public IActionResult Details(string id)
    {
        ViewBag.ProductSlug = id;
        return View();    // Views/Product/Details.cshtml
    }
}
```

Creating a default layout and enabling tag helpers

- To specify code that can be shared between multiple view files, add a Razor layout to the Views/Shared folder of your project.
- Within a Razor layout, the `ViewBag` or `ViewData` property is often used to display a title that's set in the view.
- Within a Razor layout, the `RenderBody()` method renders the body of the view. • To specify a default layout for all views, add a `_ViewStart` file to the Views folder of your project and set the `Layout` property to the name of the Razor layout.
- To enable all ASP.NET Core tag helpers for all views, add a `_ViewImports` file to the Views folder of your project that contains the code shown above

Creating a default layout and enabling tag helpers – Sample

The _Layout.cshtml file in the Views/Shared folder

```
<!DOCTYPE html>
<html>
<head>
  <title>@ViewBag.Title</title>
  <link rel="stylesheet"
        href="~/lib/bootstrap/css/bootstrap.min.css" />
  <link rel="stylesheet" href="~/css/custom.css" />
</head>
<body>
  @RenderBody()
</body>
</html>
```

A _ViewStart.cshtml file that sets the default layout

```
@{
    Layout = "_Layout";
}
```

A _ViewImports.cshtml file that enables all ASP.NET Core MVC tag helpers

```
@addTagHelper *, Microsoft.AspNetCore.Mvc.TagHelpers
```

How to add a Razor layout, view start, or view imports file

1. Right-click on the folder where you want to add the file, and select the Add→New Item item.
2. In the resulting dialog, select the ASP.NET Core→Web category.
3. Select the Razor item you want to add and respond to the resulting dialog boxes.

Using tag helpers to generate URLs for Links

Two ways to code a link

Use HTML to hard code the URL in the href attribute

```
<a href="/Product/List/Guitars">View guitars</a>
```

Use ASP.NET tag helpers to generate the URL

```
<a asp-controller="Product" asp-action="List"  
    asp-route-id="Guitars">View guitars</a>
```

The URL for both links

```
/Product/List/Guitars
```


Using tag helpers to generate URLs for Links

How to code a link to an action method in the same controller

```
<a asp-action="About">About Us</a>
```

The URL that's generated

/Home/About

How to code a link to an action method in a different controller

```
<a asp-controller="Product" asp-action="List">  
  View all products</a>
```

The URL that's generated

/Product/List

Using tag helpers to generate URLs for Links

How to code a link that includes a parameter that's in a route

```
<a asp-controller="Product" asp-action="List"
    asp-route-id="Guitars">View guitars</a>
```

The URL that's generated

```
/Product/List/Guitars
```

A link that specifies a route parameter that doesn't exist

```
<a asp-controller="Product" asp-action="List"
    asp-route-page="1" asp-route-sort_by="price">
    Products - Page 1</a>
```

The URL that's generated

```
/Product/List?page=1&sort_by=price
```

Using tag helpers to generate URLs for Links

Tag Helper	Description
<code>asp-controller</code>	Specifies the controller. This is only necessary if you want to specify a URL for an action method from another controller.
<code>asp-action</code>	Specifies the action method.
<code>asp-route-param_name</code>	Specifies a route parameter where <code>param_name</code> is the name of the parameter.

Using tag helpers to generate URLs for Links

It is considered best practice (i.e., industry standards) to use tag helpers to generate URLs

Three views that use the default layout and tag helpers

- Since the Razor code block at the top of the page doesn't set the Layout property, these views use the layout (`_Layout`) that's specified by the `_ViewStart` file.
- These views use tag helpers to specify the controller class, action method, and route id parameter for `<a>` elements.

The Home/Index view

```
@{
    ViewBag.Title = "Home";
}
<h1>Home</h1>
<div class="list-group">
    <a asp-controller="Product" asp-action="List">
        View all products</a>

    <a asp-controller="Product" asp-action="List"
        asp-route-id="Guitars">View guitars</a>

    <a asp-controller="Product" asp-action="Details"
        asp-route-id="Fender-Stratocaster">
        View Fender Stratocaster</a>
</div>
```

Three views that use the default layout and tag helpers

The Product/List view

```
@{
    ViewBag.Title = "Product List";
}
<h1>Product List</h1>
<div class="list-group">
    <p>Category: @ViewBag.Category</p>

    <a asp-action="Details"
       asp-route-id="Fender-Stratocaster">
        View Fender Stratocaster</a>

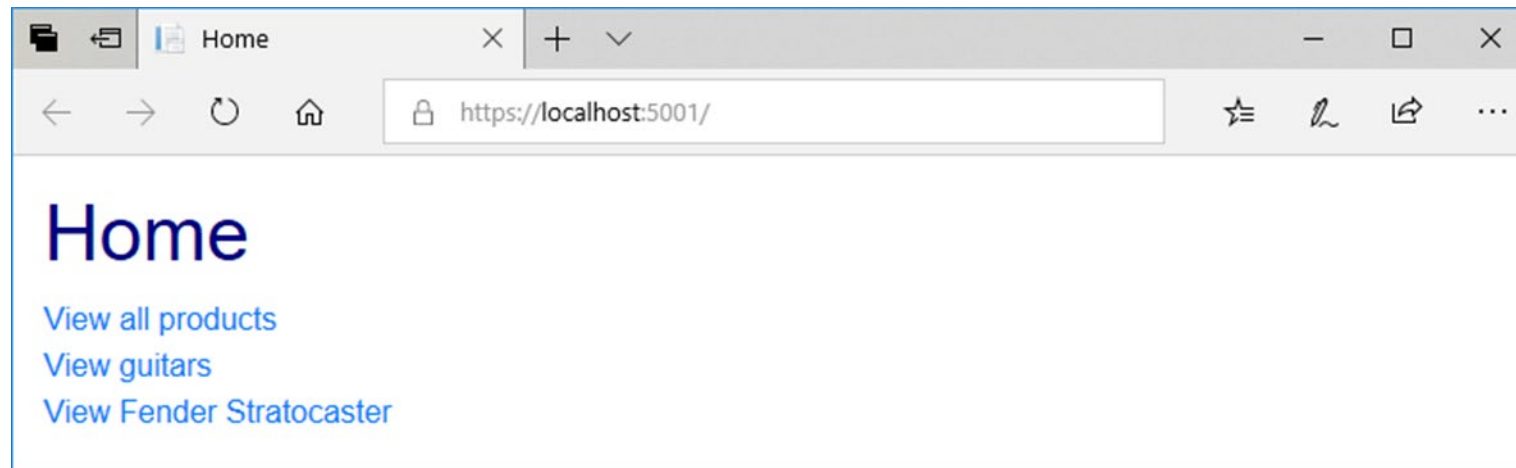
    <a asp-controller="Home" asp-action="Index">Home</a>
</div>
```

The Product/Details view

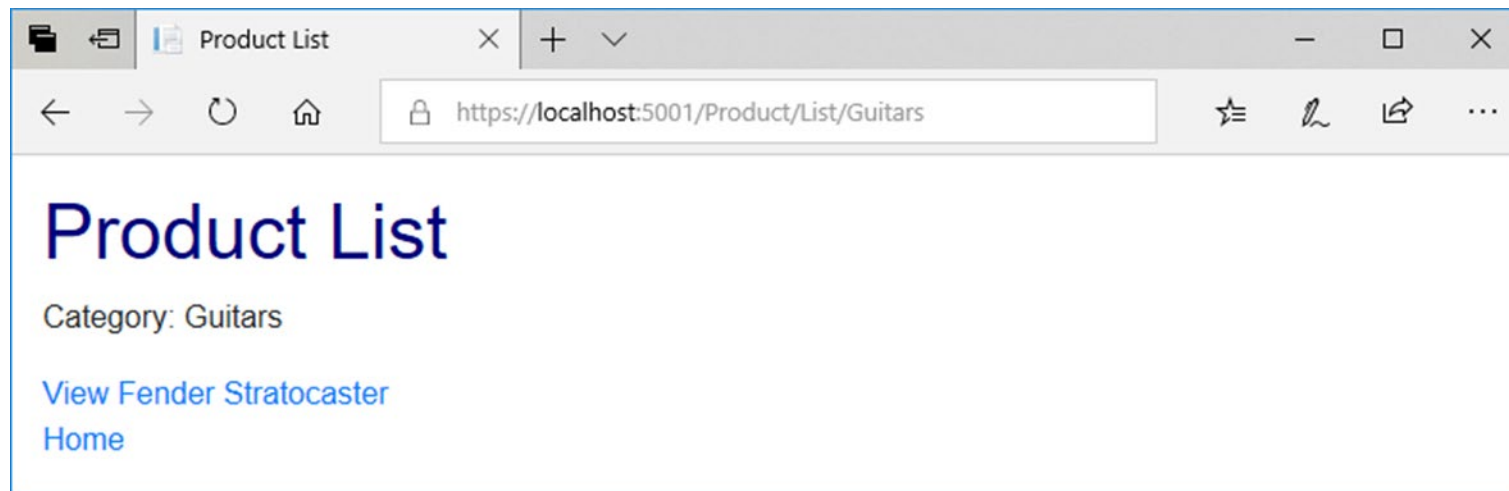
```
@{
    ViewBag.Title = "Product Details";
}
<h1>Product Details</h1>
<div class="list-group">
    <p>Slug: @ViewBag.ProductSlug</p>

    <a asp-controller="Home" asp-action="Index">Home</a>
</div>
```

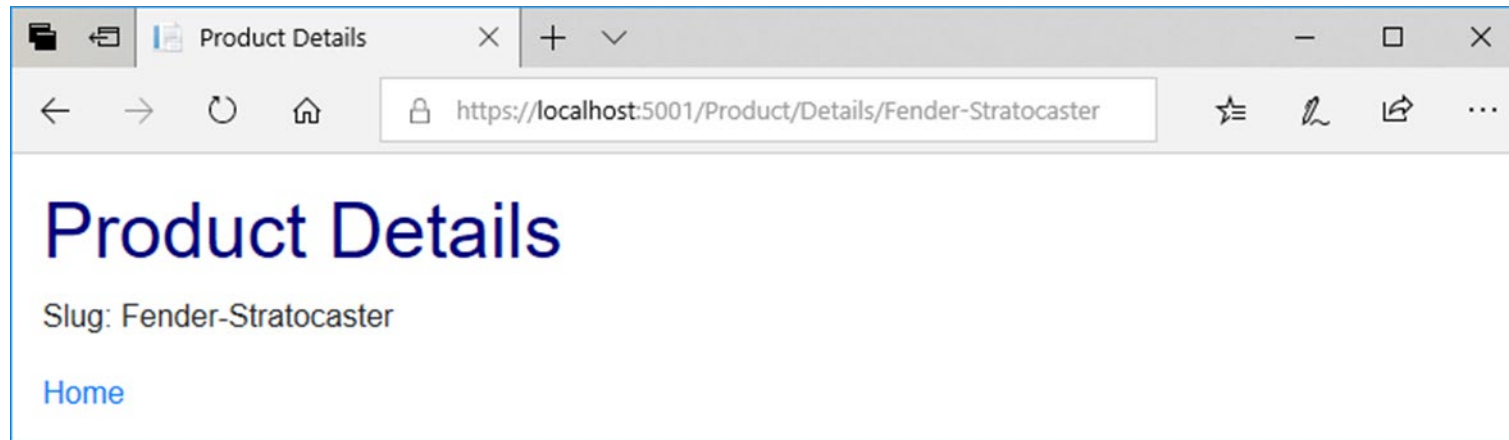
The three views displayed in a browser



The three views displayed in a browser



The three views displayed in a browser



More tag helpers for generating URLs for links

- A URL **fragment** allows you to jump to a specified placeholder on a web page. A placeholder is identified by an id attribute, and a fragment is preceded by the hash mark (#) in a URL.
- A **relative URL** is a URL that's relative to the app's root directory.
- An **absolute URL** is a URL that specifies the host.
- Relative URLs are more flexible than absolute URLs because they allow your app to work on multiple hosts. As a result, you should only use an absolute URL when coding a link to a specific host outside your app
- The core method that would be useful for your assignments is the `asp-area` tag helper – which specifies the area for the URL (see last week's lecture).

More tag helpers for generating URLs for links

- The core method that would be useful for your assignments is the `asp-area` tag helper – which specifies the area for the URL (see last week's lecture).

How to code a link to an area

```
<a asp-area="Admin" asp-controller="Product"  
    asp-action="List">Admin - Product Manager</a>
```

The URL that's generated

```
/Admin/Product/List
```

Working with placeholders

How to code an HTML placeholder

```
<h2 id="Fender">Fender Guitars</h2>
```

How to code a URL that jumps to an HTML placeholder on the same page

```
<a asp-fragment="Fender">View Fender Guitars</a>
```

The URL that's generated

```
/ #Fender
```

Creating an absolute URL

How to code an absolute URL

```
<a asp-protocol="https"  
    asp-host="murach.com"  
    asp-controller="Shop"  
    asp-action="Details"  
    asp-route-id="html5-and-css3"  
    asp-fragment="reviews">Murach's HTML5 and CSS3 - Reviews</a>
```

The URL that's generated

```
https://www.murach.com/Shop/Details/html5-and-css3#reviews
```

Formatting numbers in Views

- In a Razor expression, you can use C#'s format specifiers to format a number with thousandths separators, as currency, or as a percent. One way to do that is to call the `ToString()` method from the numeric data type and pass it the format specifier.
- By default, the format specifiers return two decimal places, but you can specify a different number.
- If necessary, the format specifiers round numbers to the nearest decimal place.
- This figure presents three of the most common format specifiers, but many others exist. For more information, you can search the internet.

Formatting numbers in Views

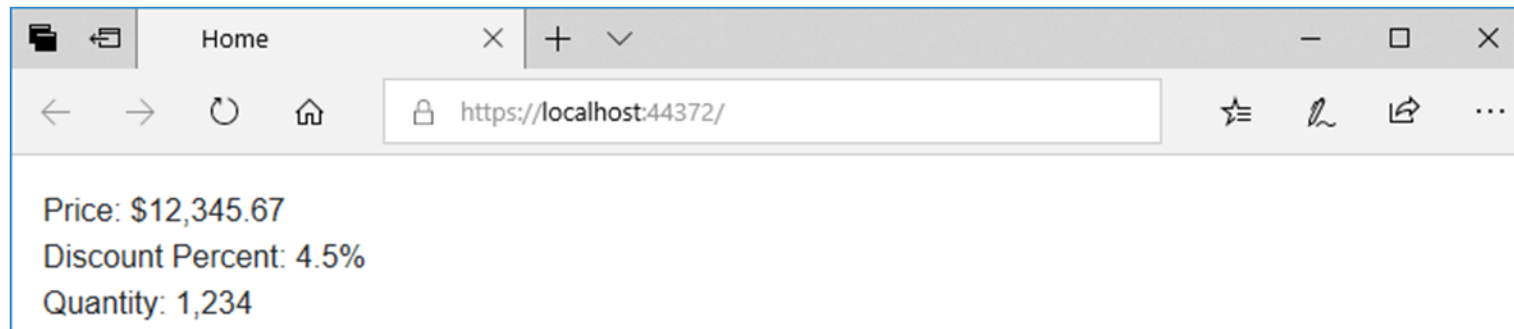
Expression	Result
<code>@ViewBag.Price.ToString("C")</code>	\$12,345.67
<code>@ViewBag.Price.ToString("C1")</code>	\$12,345.7
<code>@ViewBag.Price.ToString("C0")</code>	\$12,346
<code>@ViewBag.Price.ToString("N")</code>	12,345.67
<code>@ViewBag.Price.ToString("N1")</code>	12,345.7
<code>@ViewBag.Price.ToString("N0")</code>	12,346
<code>@ViewBag.DiscountPercent.ToString("P")</code>	4.50%
<code>@ViewBag.DiscountPercent.ToString("P1")</code>	4.5%
<code>@ViewBag.DiscountPercent.ToString("P0")</code>	5%

Formatting numbers in Views

Code in a view that displays the numbers

```
<div>Price: @ViewBag.Price.ToString("C")</div>  
<div>Discount Percent: @ViewBag.DiscountPercent.ToString("P1")</div>  
<div>Quantity: @ViewBag.Quantity.ToString("N0")</div>
```

The view displayed in a browser



Passing a Model to a View

- A **model** is a regular C# class that defines an object.
- In a controller, you can use the `View()` method to pass a model object to a view. Then, an action method can return the `ViewResult` object.
- In this figure, the context variable refers to a `DbContext` object that provides access to the data for the app as described in the Week 3 Lecture

The Product model

```
namespace GuitarShop.Models
{
    public class Product
    {
        public int ProductID { get; set; }
        public Category Category { get; set; }
        public string Name { get; set; }
        public decimal Price { get; set; }
        public string Slug => Name.Replace(' ', '-');
    }
}
```

Passing a Model to a View

A method that a controller can use to pass a model to a view

```
View(model)
View(name, model)
```

The Product/Details() action method that passes a model to a view

```
public IActionResult Details(int id) {
    Product product = context.Products.Find(id);
    return View(product);
}
```

A Product/Add() action method that passes a model to the specified view

```
[HttpGet]
public IActionResult Add() {
    return View("Update", new Product());
}
```

How to display model properties in a View

- In a view, you can use the `@model` directive to bind a model to a view.
- After that, you can use the `Model` property to access the properties and methods of the model object.

A `_ViewImports` file that imports the Models namespace for all views

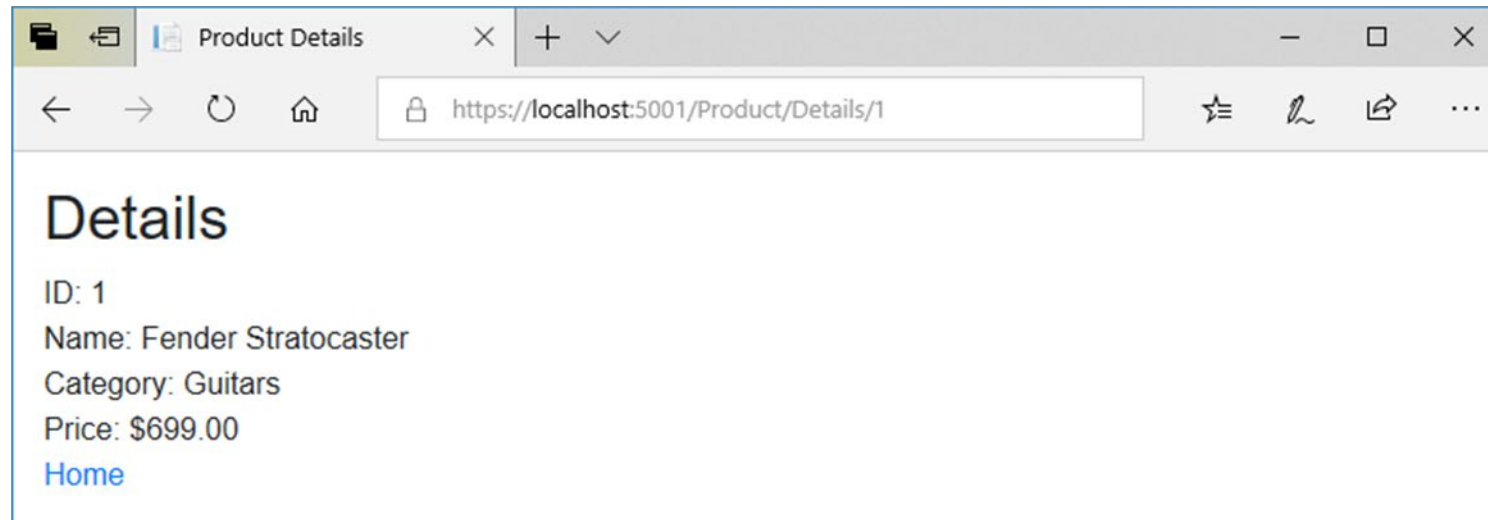
```
@addTagHelper *, Microsoft.AspNetCore.Mvc.TagHelpers
@using GuitarShop.Models
```

The code for the `Product/Details` view

```
@model Product
@{
    ViewBag.Title = "Product Details";
}
<h1>Product Details</h1>
<div>ID: @Model.ProductID</div>
<div>Name: @Model.Name</div>
<div>Category: @Model.Category.Name</div>
<div>Price: @Model.Price.ToString("C2")</div>
<a asp-controller="Home" asp-action="Index">Home</a>
```

How to display model properties in a View

The view with model properties in a browser



Binding model properties to HTML Elements

- Use the `asp-for` tag helper to bind models' properties to HTML element like the `<label>` or `<input>` elements

A tag helper for binding HTML elements to model properties

`asp-for`

A view with `asp-for` tag helpers (part 1)

```
@model Product
@{
    ViewBag.Title = "Update Product";
}
<h1>Update Product</h1>
<form asp-action="Update" method="post">
    <div class="form-group">
        <label asp-for="Name"></label>
        <input asp-for="Name" class="form-control">
    </div>

    <div class="form-group">
        <label asp-for="Price"></label>
        <input asp-for="Price" class="form-control">
    </div>
</form>
```

Binding model properties to HTML Elements

- Use the `asp-for` tag helper to bind models' properties to HTML element like the `<label>` or `<input>` elements

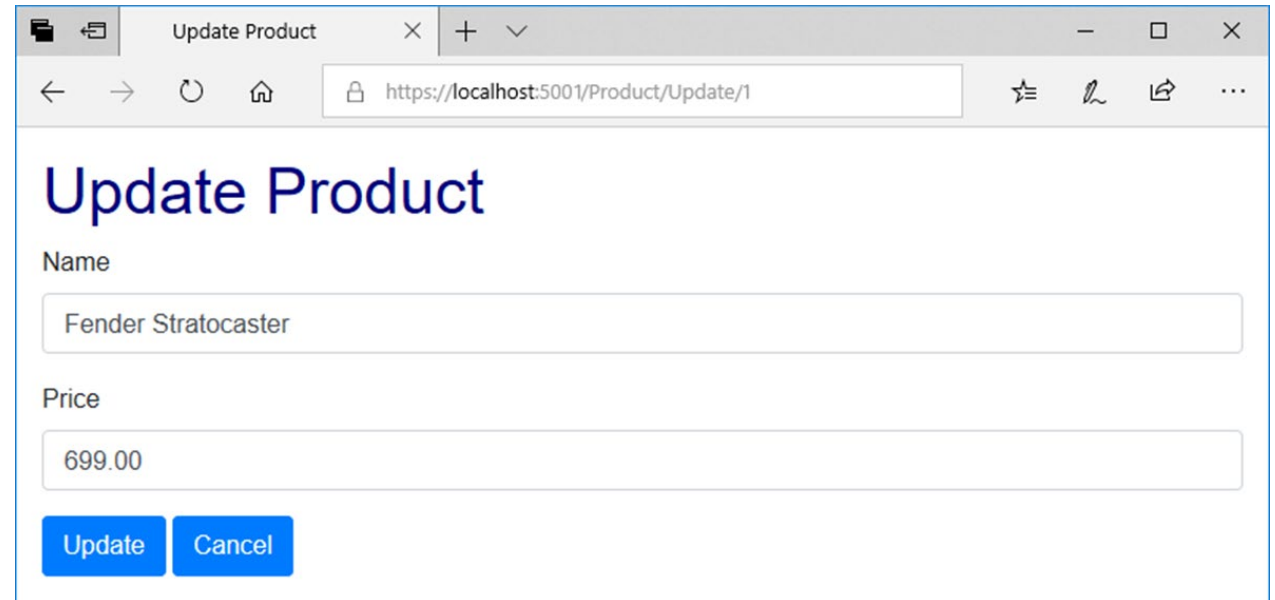
A view with `asp-for` tag helpers (part 2)

```
<input type="hidden" asp-for="ProductID" />
<input type="hidden" asp-for="Category.Name" />

<button asp-action="Update" type="submit"
        class="btn btn-primary">Update
</button>
<a asp-action="List" class="btn btn-primary">
    Cancel</a>
</form>
```

Binding model properties to HTML Elements

- Use the `asp-for` tag helper to bind models' properties to HTML element like the `<label>` or `<input>` elements



Update Product

Name

Fender Stratocaster

Price

699.00

Update Cancel

The HTML that's generated for the Price `<label>` and `<input>` elements

```
<label for="Price">Price</label>
<input class="form-control" type="text"
       id="Price" name="Price" value="699.00">
```

Binding a list of items to a <select> element

- You can use the `asp-items` tag helper to bind items to a `<select>` element. This creates the `<option>` elements for the `<select>` element.
- In this figure, the context variable refers to a `DbContext` object that provides access to the data for the app as described in the Week 3 Lecture

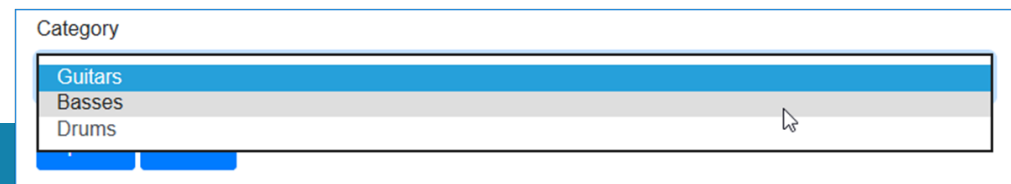
An Update action method that creates the model and passes it to the view

```
public IActionResult Update(int id)
{
    ViewBag.Categories = context.Categories.ToList();
    Product product = context.Products.Find(id);
    return View(product);
}
```

The code that binds items to a <select> element

```
<div class="form-group">
    <label asp-for="CategoryID">Category</label>
    <select asp-for="CategoryID"
        asp-items="@ (new SelectList(ViewBag.Categories,
            "CategoryID", "Name",
            Model.CategoryID.ToString())) "
        class="form-control"></select>
</div>
```

The <select> element and its label displayed in a browser



The screenshot shows a web browser window with a form. At the top, there is a label 'Category'. Below the label is a dropdown menu (select element) with three visible options: 'Guitars', 'Basses', and 'Drums'. The 'Guitars' option is currently selected and highlighted in blue. A mouse cursor is visible over the dropdown menu.

Binding a list of items to a `<select>` element

- You can use the `asp-items` tag helper to bind items to a `<select>` element. This creates the `<option>` elements for the `<select>` element.
- In this figure, the context variable refers to a `DbContext` object that provides access to the data for the app as described in the Week 3 Lecture

HTML that's generated for the `<select>` element

```
<select class="form-control" id="Category_CategoryID"
        name="Category.CategoryID">
    <option selected="selected" value="1">
        Guitars</option>
    <option value="2">Basses</option>
    <option value="3">Drums</option>
</select>
```

Displaying a list of Model Objects

A List action method that creates the model and passes it to the view

```
public IActionResult List(string id = "All")
{
    ViewBag.Categories = context.Categories.ToList();
    ViewBag.SelectedCategory = id;
    List<Product> products = context.Products
        .Where(p => p.Category.Name == id).ToList();
    return View(products);
}
```

Sample View Code for displaying a list of Model Objects

Part 1

```
@model List<Product>
@{
    ViewBag.Title = "Product List";
}

<h1>Product List</h1>
@if (ViewBag.Categories != null)
{
    foreach (Category c in ViewBag.Categories)
    {
        <a asp-controller="Product" asp-action="List"
            asp-route-id="@c.Name">@c.Name</a>
        <text> | </text>
    }
}
<a asp-controller="Product" asp-action="List"
    asp-route-id="All">All</a>
<hr />
```

Part 2

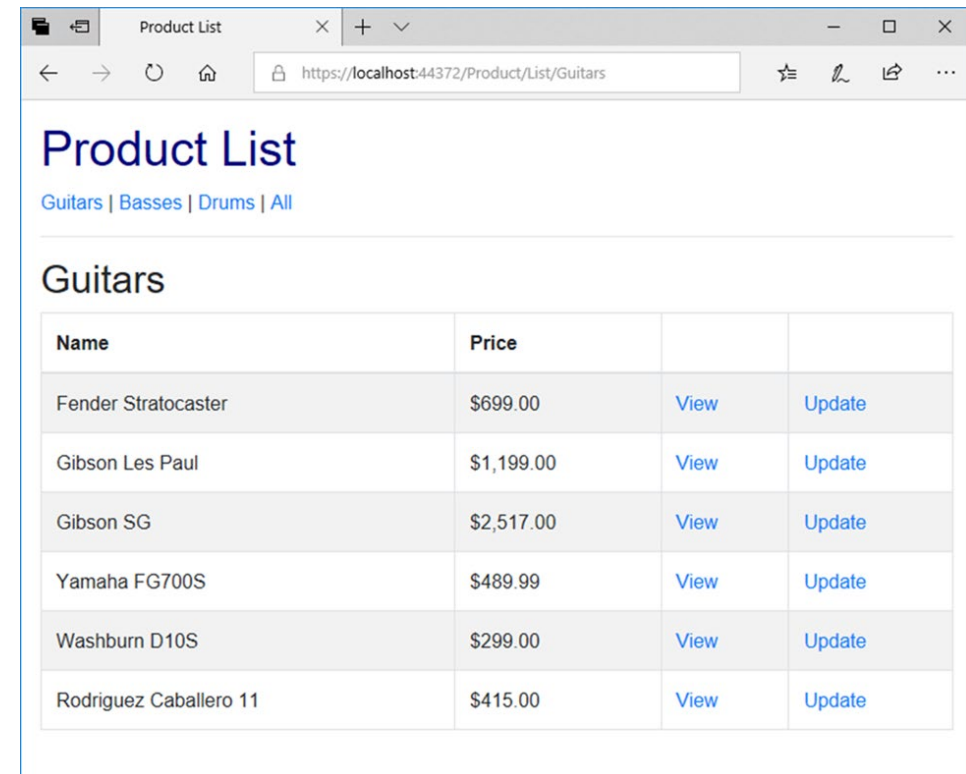
```
@if (@ViewBag.SelectedCategory == "All")
{
    <h2>All Products</h2>
}
else
{
    <h2>@ViewBag.SelectedCategory</h2>
}
<table class="table table-bordered table-striped">
    <thead>
        <tr>
            <th>Name</th>
            <th>Price</th>
            <th></th>
            <th></th>
        </tr>
    </thead>
```

Sample View Code for displaying a list of Model Objects

Part 3

```
<tbody>
  @foreach (var product in @Model)
  {
    <tr>
      <td>@product.Name</td>
      <td>@product.Price.ToString("C")</td>
      <td>
        <a asp-controller="Product"
           asp-action="Details"
           asp-route-id="@product.ProductID">
          View</a>
      </td>
      <td>
        <a asp-controller="Product"
           asp-action="Update"
           asp-route-id="@product.ProductID">
          Update</a>
      </td>
    </tr>
  }
</tbody>
</table>
```

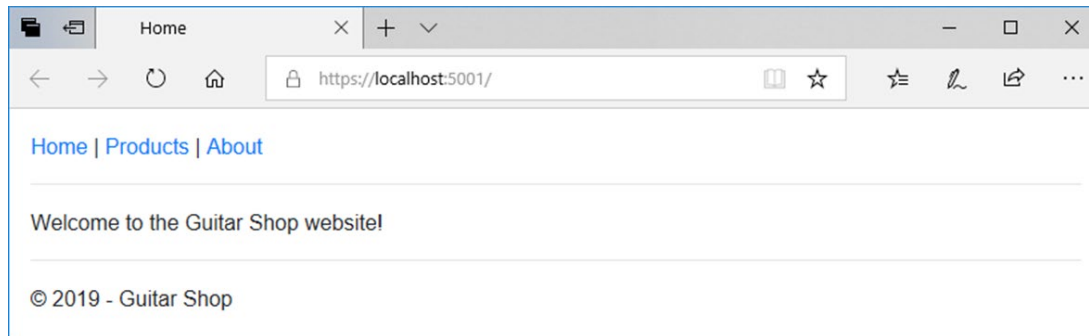
The Resulting View



Name	Price		
Fender Stratocaster	\$699.00	View	Update
Gibson Les Paul	\$1,199.00	View	Update
Gibson SG	\$2,517.00	View	Update
Yamaha FG700S	\$489.99	View	Update
Washburn D10S	\$299.00	View	Update
Rodriguez Caballero 11	\$415.00	View	Update

Creating and Applying a Layout

- A project can contain multiple Razor layouts. That way, different views can use different layouts.



The code for a `_ViewStart.cshtml` file that sets the default layout

```
@{  
    Layout = "_Layout";  
}
```

The code for a Home/Index view that explicitly specifies a layout

```
@{  
    Layout = "_MainLayout";  
    ViewBag.Title = "Home";  
}  
<p>Welcome to the Guitar Shop website!</p>
```

Creating and Applying a Layout

The Views/Shared/_MainLayout.cshtml file (part 1)

```
<!DOCTYPE html>
<html>
<head>
  <title>@ViewBag.Title</title>
  <link rel="stylesheet"
href="~/lib/bootstrap/css/bootstrap.min.css" />
  <link rel="stylesheet" href="~/css/custom.css" />
</head>
<body>
  <header>
    <a asp-controller="Home" asp-action="Index">
      Home</a> |
    <a asp-controller="Product" asp-action="List">
      Products</a> |
    <a asp-controller="Home" asp-action="About">
      About</a>
    <hr />
  </header>
```

The Views/Shared/_MainLayout.cshtml file (part 2)

```
@RenderBody()

<footer>
  <hr />
  <p>&copy; @DateTime.Now.Year - Guitar Shop</p>
</footer>
</body>
</html>
```

Nesting Layouts

- To avoid code duplication, you can build layouts by nesting one layout within another.

The _Layout file

```
<!DOCTYPE html>
<html>
<head>
  <title>@ViewBag.Title</title>
  <link rel="stylesheet"
        href="~/lib/bootstrap/css/bootstrap.min.css" />
  <link rel="stylesheet" href="~/css/custom.css" />
</head>
<body>
  @RenderBody()
</body>
</html>
```

Nesting Layouts

- To avoid code duplication, you can build layouts by nesting one layout within another.

The `_MainLayout` file

```
@{  
    Layout = "_Layout";  
}  
  
<header>  
    <a asp-controller="Home" asp-action="Index">Home</a> |  
    <a asp-controller="Product" asp-action="List"  
        asp-route-id="All">Products</a> |  
    <a asp-controller="Home" asp-action="About">About</a>  
    <hr />  
</header>  
  
@RenderBody()  
  
<footer>  
    <hr />  
    <p>&copy; @DateTime.Now.Year - Guitar Shop</p>  
</footer>
```


Nesting Layouts

- To avoid code duplication, you can build layouts by nesting one layout within another.

The `_ProductLayout` file

```
@{
    Layout = "_MainLayout";
}

<h1>Product Manager</h1>
@if (ViewBag.Categories != null)
{
    foreach (Category c in ViewBag.Categories)
    {
        <a asp-controller="Product" asp-action="List"
            asp-route-id="@c.Name">@c.Name</a><text> | </text>
    }
    <a asp-controller="Product" asp-action="List"
        asp-route-id="All">All</a>
}
<hr />

@RenderBody()
```

Nesting Layouts

- To avoid code duplication, you can build layouts by nesting one layout within another.

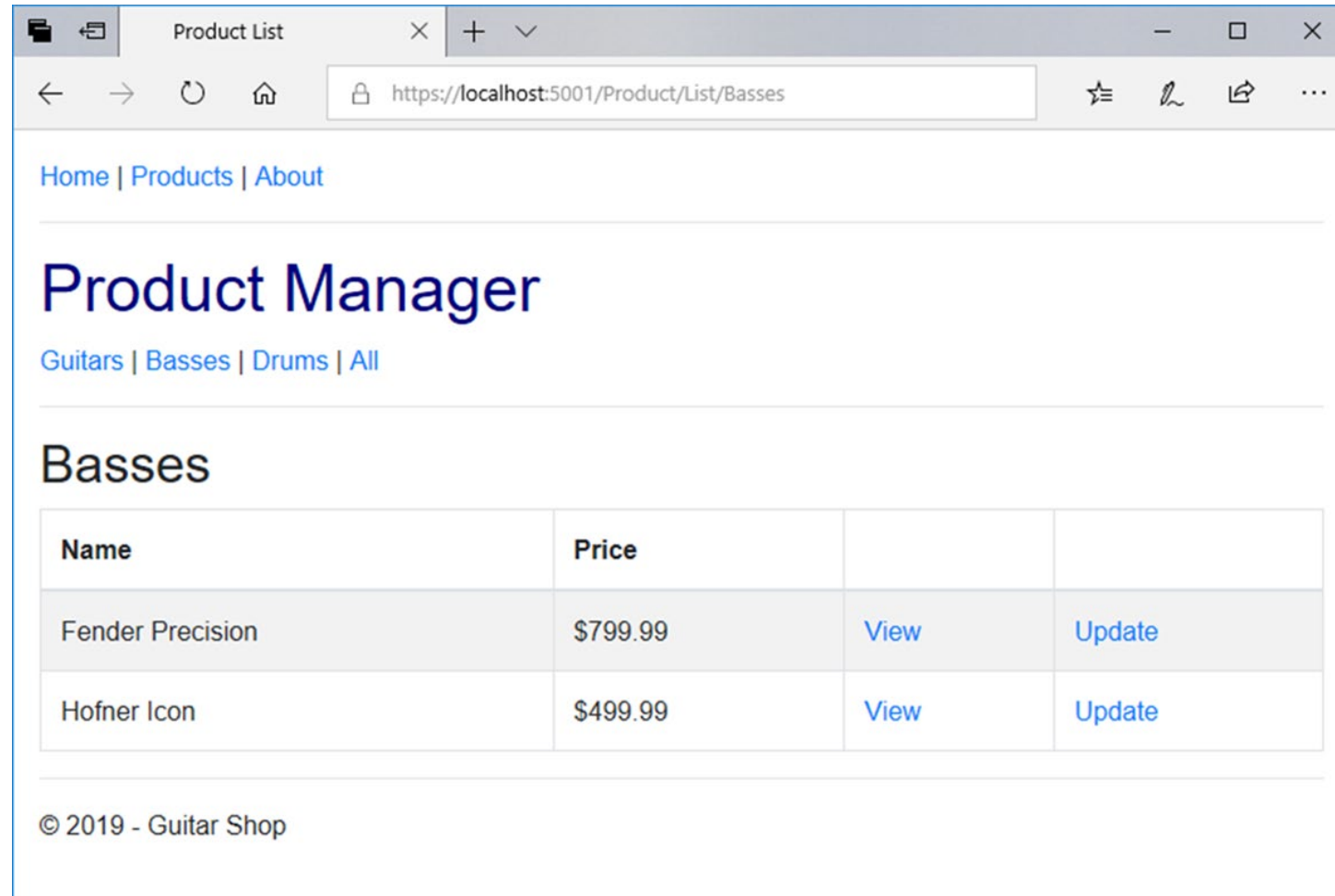
The code for a view that uses the `_ProductLayout` view

```
@model List<Product>
@{
    Layout = "_ProductLayout";
    ViewBag.Title = "Product List";
}

@if (@ViewBag.SelectedCategory == "All")
{
    <h2>All Products</h2>
}
else
{
    <h2>@ViewBag.SelectedCategory</h2>
}

<!-- The code for the <table> element -->
```

The Result – Example Nested Layout



Other Nested Examples

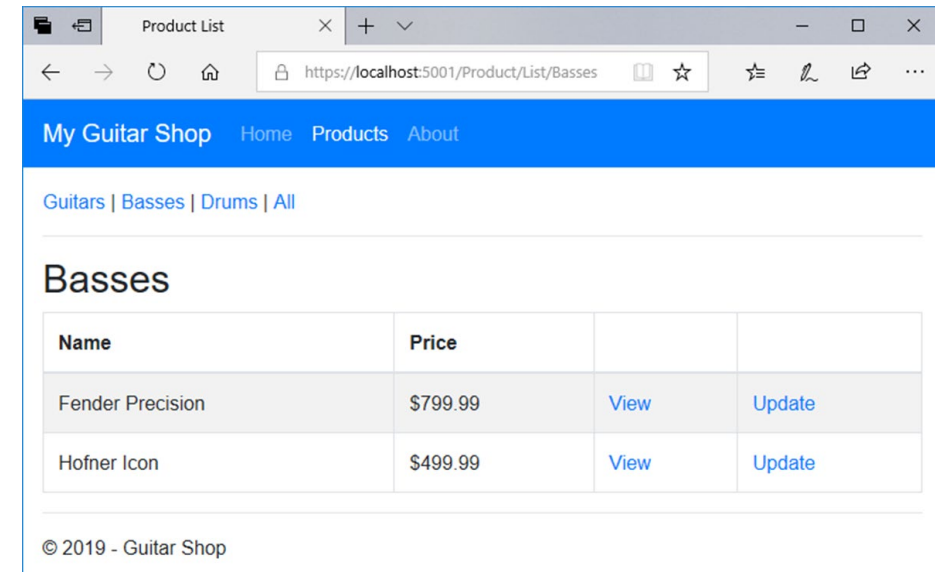
A _MainLayout file that uses view context (part 1)

```
@{
    Layout = "_Layout";
    string controller =
        ViewContext.RouteData.Values["controller"].ToString();
    string action =
        ViewContext.RouteData.Values["action"].ToString();
}
<nav class="navbar navbar-dark bg-primary fixed-top">
    <a class="navbar-brand" href="/">My Guitar Shop</a>
    <div class="navbar-nav">
        <a class="nav-link"
            @(controller == "Home" && action == "Index" ?
               "active" : "")
            asp-controller="Home" asp-action="Index">Home</a>
        <a class="nav-link"
            @(controller == "Product" ? "active" : "")
            asp-controller="Product" asp-action="List">
            Products</a>
        <a class="nav-link"
            @(controller == "Home" && action == "About" ?
               "active" : "")
            asp-controller="Home" asp-action="About">About</a>
    </div>
</nav>
```

A _MainLayout file that uses view context (part 2)

```
@RenderBody()

<footer>
    <hr />
    <p>&copy; @DateTime.Now.Year - Guitar Shop</p>
</footer>
```



Using Sections

- Within a layout, you can use the `RenderSection()` method to specify where content from a section can be inserted into the layout. A section is a block of content that has a name.
- Within a view, you can use the Razor `@section` declaration to specify a section that will be inserted into the designated location in the layout.
- You can also code an `@section` declaration that contains a `RenderSection()` method in a nested layout. Then, the code in that section is available to any layout that uses the nested layout and that also includes a `RenderSection()` method for the section

Using Sections

Code in a view file that specifies a section

```
@model Product
@{
    ViewBag.Title = "Update Product";
}

@section scripts {
    <script src="~/lib/jquery-validate/jquery.validate.min.js">
    </script>
    <script src="~/lib/jquery-validation-unobtrusive/
        jquery.validate.unobtrusive.min.js">
    </script>
}

<h2>Update</h2>

// the HTML elements for the rest of the view body go here
```

A method that can insert content from a section into a layout

```
RenderSection(name, isRequired)
```

A layout file that specifies an optional section

```
<!DOCTYPE html>
<html>
<head>
    <title>@ViewBag.Title</title>
    <link rel="stylesheet"
        href="~/lib/bootstrap/css/bootstrap.min.css" />
    <link rel="stylesheet" href="~/css/custom.css" />

    <script src="~/lib/jquery/jquery.min.js"></script>
    <script src="~/lib/popper.js/popper.min.js"></script>
    <script src="~/lib/bootstrap/js/bootstrap.min.js"></script>
    @RenderSection("scripts", false)
</head>
<body>
    @RenderBody()
</body>
</html>
```

Other Nested Examples – Customer Product List and Details Page

Product List - Guitar Shop

https://localhost:5001/Products/Guitars

My Guitar Shop Home Products About Admin

Product List

All Guitars Bases Drums

Name	Price		
Fender Stratocaster	\$699.00	View Details	Add to Cart
Gibson Les Paul	\$1,199.00	View Details	Add to Cart
Gibson SG	\$2,517.00	View Details	Add to Cart
Yamaha FG700S	\$489.99	View Details	Add to Cart
Washburn D10S	\$299.00	View Details	Add to Cart
Rodriguez Caballero 11	\$415.00	View Details	Add to Cart

© 2019 - Guitar Shop

Product Details - Guitar Shop

https://localhost:5001/Product/Details/2/Gibson-Les-Paul

My Guitar Shop Home Products About Admin

Product Details



Gibson Les Paul

List Price:	\$1,199.00
Discount Percent:	20%
Your Price:	\$959.20 (You save \$239.80)

[Add to Cart](#)

© 2019 - Guitar Shop

Other Nested Examples – The Product Manager page for administrators

Product Manager - Guitars

https://localhost:5001/Admin/Products/Guitars

My Guitar Shop Admin Products Categories View Site

Product Manager

[All](#)
[Guitars](#)
[Basses](#)
[Drums](#)

Name	Price		
Fender Stratocaster	\$699.00	Update	Delete
Gibson Les Paul	\$1,199.00	Update	Delete
Gibson SG	\$2,517.00	Update	Delete
Yamaha FG700S	\$489.99	Update	Delete
Washburn D10S	\$299.00	Update	Delete
Rodriguez Caballero 11	\$415.00	Update	Delete

[Add New Product](#)

© 2019 - Guitar Shop

Update Product - Guitars

https://localhost:5001/Admin/Product/Update/2

My Guitar Shop Admin Products Categories View Site

Update Product

Category
Guitars

Code
les_paul

Name
Gibson Les Paul

Price
1199.00

[Update](#) [Cancel](#)

© 2019 - Guitar Shop

Other Nested Examples – The Product Manager page for administrators

Product Manager

My Guitar Shop Admin Products Categories View Site

All Guitars Bases Drums

Name	Price		
Fender Stratocaster	\$699.00	Update	Delete
Gibson Les Paul	\$1,199.00	Update	Delete
Gibson SG	\$2,517.00	Update	Delete
Yamaha FG700S	\$489.99	Update	Delete
Washburn D10S	\$299.00	Update	Delete
Rodriguez Caballero 11	\$415.00	Update	Delete

Add New Product

© 2019 - Guitar Shop

Add Product

My Guitar Shop Admin Products Categories View Site

Category
Guitars

Code
test

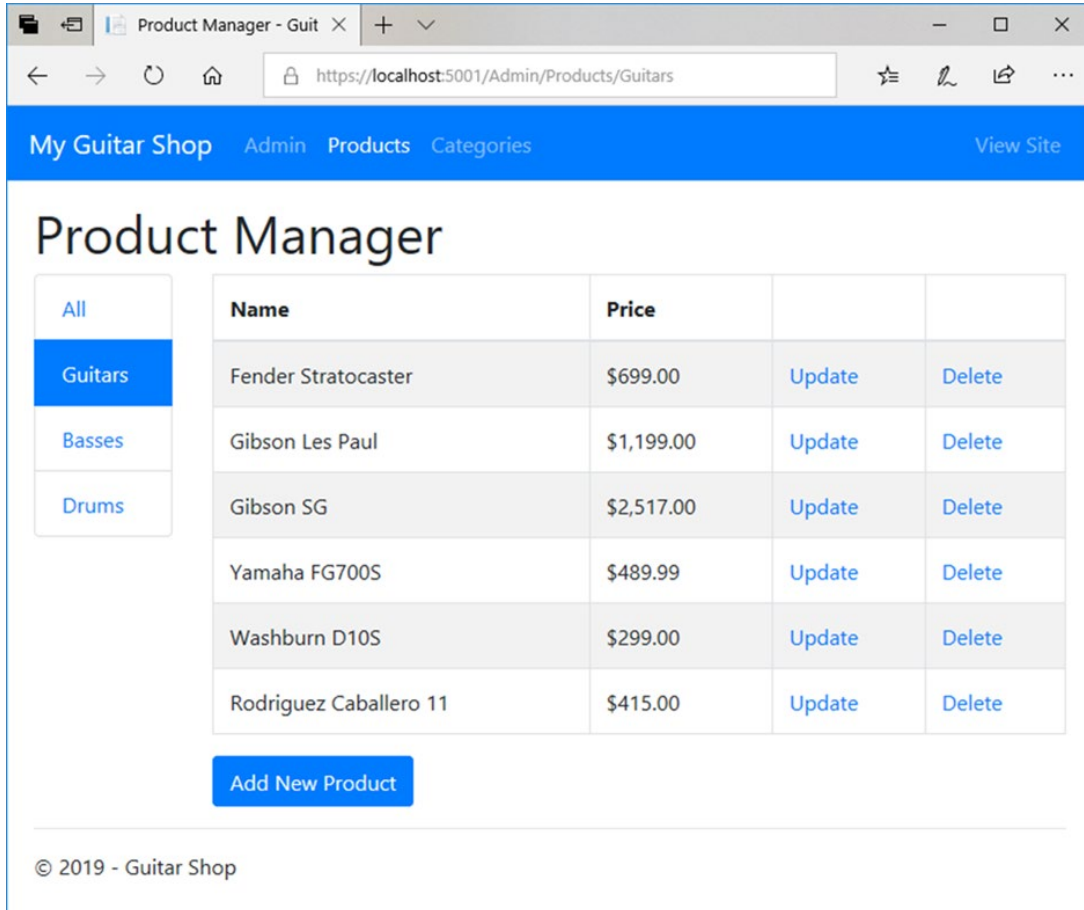
Name
Test

Price
650

Add Cancel

© 2019 - Guitar Shop

Other Nested Examples – The Product Manager page for administrators



Product Manager - Guit

https://localhost:5001/Admin/Products/Guitars

My Guitar Shop Admin Products Categories View Site

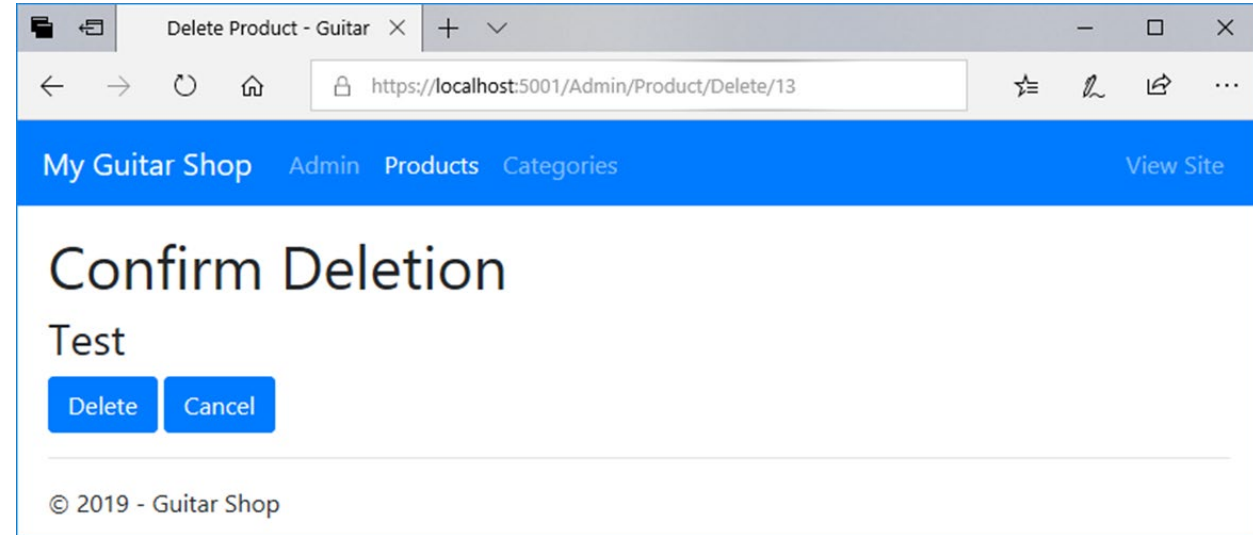
Product Manager

All Guitars Basses Drums

Name	Price		
Fender Stratocaster	\$699.00	Update	Delete
Gibson Les Paul	\$1,199.00	Update	Delete
Gibson SG	\$2,517.00	Update	Delete
Yamaha FG700S	\$489.99	Update	Delete
Washburn D10S	\$299.00	Update	Delete
Rodriguez Caballero 11	\$415.00	Update	Delete

Add New Product

© 2019 - Guitar Shop



Delete Product - Guitar

https://localhost:5001/Admin/Product/Delete/13

My Guitar Shop Admin Products Categories View Site

Confirm Deletion

Test

Delete Cancel

© 2019 - Guitar Shop



How to work with
tag helpers,
partial views and
view components

Common built-in tag helpers

Tag helper	Description
<code>asp-action</code>	Attribute that indicates the action method in a route.
<code>asp-controller</code>	Attribute that indicates the controller in a route.
<code>asp-area</code>	Attribute that indicates the area in a route.
<code>asp-for</code>	Attribute that binds an element to a property of a model object. MVC will generate the appropriate attributes for the element.
<code>asp-validation-summary</code>	Attribute that controls the display of validation messages. Usually applied to a <code><div></code> tag.
<code>asp-validation-for</code>	Attribute that controls the display of a single validation message. Usually applied to a <code></code> tag.
<code>environment</code>	Element that uses <code>include</code> and <code>exclude</code> attributes to display HTML based on the current hosting environment.
<code>partial</code>	Element that renders a partial view.

Creating custom Tag helpers

- To create a custom tag helper, you code a class that inherits the `TagHelper` class, and you typically override its virtual `Process()` method.
- A tag helper class automatically applies to the HTML element of the same name.
- By convention, tag helper classes have a suffix of `TagHelper`, but this is not required.
- The `TagHelperContext` class represents the current context of an HTML element.
- The `TagHelperOutput` class represents the state of the HTML element.

How to create a tag helper for a non-standard HTML element

- MVC automatically translates between the Pascal casing of a C# class name such as `SubmitButton` and the kebab casing of the corresponding HTML element such as `<submit-button>`

A tag helper that applies to any non-standard `<submit-button>` element

```
public class SubmitButtonTagHelper : TagHelper
{
    public override void Process(TagHelperContext context,
        TagHelperOutput output)
    {
        // make it a button element with start and end tags
        output.TagName = "button";
        output.TagMode = TagMode.StartTagAndEndTag;

        // make it a submit button
        output.Attributes.SetAttribute("type", "submit");

        // append bootstrap button classes
        string newClasses = "btn btn-primary";
        string oldClasses =
            output.Attributes["class"]?.Value?.ToString();
        string classes = (string.IsNullOrEmpty(oldClasses)) ?
            newClasses : $"{oldClasses} {newClasses}";
        output.Attributes.SetAttribute("class", classes);
    }
}
```

How to use extension methods with tag helpers

Three extension methods for tag helpers (part 1)

```
using Microsoft.AspNetCore.Razor.TagHelpers;
...
public static class TagHelperExtensions
{
    public static void AppendCssClass(
        this TagHelperAttributeList list, string newCssClasses)
    {
        string oldCssClasses = list["class"]?.Value?.ToString();
        string cssClasses = (string.IsNullOrEmpty(oldCssClasses)) ?
            newCssClasses : $"{oldCssClasses} {newCssClasses}";
        list.SetAttribute("class", cssClasses);
    }

    public static void BuildTag(this TagHelperOutput output,
        string tagName, string classNames)
    {
        output.TagName = tagName;
        output.TagMode = TagMode.StartTagAndEndTag;
        output.Attributes.AppendCssClass(classNames);
    }
}
```

Three extension methods for tag helpers (part 2)

```
public static void BuildLink(this TagHelperOutput output,
    string url, string className)
{
    output.BuildTag("a", className);
    output.Attributes.SetAttribute("href", url);
}
```

How to control the scope of a tag helper

- You can use the `HTMLTargetElement` attribute to...
 - Allow a tag helper class to have a different name than the HTML element it targets.
 - Narrow the scope of a tag helper so it only targets an element under certain conditions.
 - Widen the scope of a tag helper so it targets multiple elements.
- The `HtmlTargetElement` attribute specifies the target HTML element for a tag helper.
- You can apply multiple `HtmlTargetElement` attributes to a tag helper class. • Within the `HtmlTargetElement` attribute, you can use brackets to specify the name and value of an attribute.

How to control the scope of a tag helper

A tag helper for any <label> element

```
public class LabelTagHelper : TagHelper {...}
```

Another tag helper for any <label> element

```
[HtmlTargetElement("label")]  
public class MyLabelTagHelper : TagHelper {...}
```

A tag helper for bound <label> elements in a form

```
[HtmlTargetElement("label", Attributes = "asp-for",  
ParentTag = "form")]  
public class FormLabelTagHelper : TagHelper {...}
```

A tag helper for <input> or <select> elements in a form

```
[HtmlTargetElement("input", ParentTag = "form")]  
[HtmlTargetElement("select", ParentTag = "form")]  
public class FormTagHelper : TagHelper {...}
```

How to use a tag helper to add elements

- The properties of the `TagHelperOutput` class return a type of `TagHelperContent` that provides methods for getting and setting the HTML that's output to the browser

A tag helper that adds a `<span>` element after the targeted element

```
[HtmlTargetElement("input", Attributes = "my-required")]
public class RequiredInputTagHelper : TagHelper
{
    public override void Process(TagHelperContext context,
        TagHelperOutput output)
    {
        // add CSS class to input element
        output.Attributes.AppendCssClass("form-control");

        // create a <span> element
        TagBuilder span = new TagBuilder("span");
        span.Attributes.Add("class", "text-danger mr-2");
        span.InnerHtml.Append("*");

        // add span element after input element
        output.PostElement.AppendHtml(span);
    }
}
```

How to use properties with a tag helper

- You can use properties in a tag helper class to get data from the view.
- You can use the `HtmlAttributeName` attribute to define the name of the HTML attribute if you want it to be different from the property name

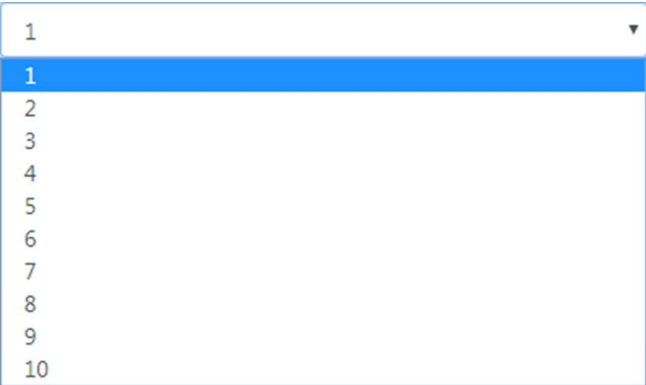
A view that uses the `NumberDropDown` tag helper

```
@model CartItem
...
<select asp-for="Quantity" class="form-control"
        my-min-number="1" my-max-number="10"></select>
```

The HTML that's sent to the browser

```
<select class="form-control" id="Quantity"
name="Quantity">
  <option>1</option>
  <option>2</option>
  ...
  <option>10</option>
</select>
```

The `<select>` element displayed in a browser



A screenshot of a web browser displaying a dropdown menu. The menu is open, showing a list of numbers from 1 to 10. The number '1' is selected and highlighted in blue. The dropdown menu has a small arrow icon in the top right corner.

How to work with the model property that an element is bound to

- The `ModelExpression` class represents a model property that's bound to an element with the `asp-for` tag helper.
- You can use the `AllAttributes` property of the `TagHelperContext` class to retrieve a `ModelExpression` object for a bound element.

How to use ViewContext & dependency injection with a tag helper

- If you create a property of the `ViewContext` type and decorate it with the `ViewContext` attribute, MVC automatically sets the value of that property to the value of the `ViewContext` object.
- The `HtmlAttributeNotBound` attribute tells MVC that a property isn't set in the HTML.
- You can use dependency injection to inject a custom object into a tag helper.

How to use ViewContext & dependency injection with a tag helper

- If you create a property of the `ViewContext` type and decorate it with the `ViewContext` attribute, MVC automatically sets the value of that property to the value of the `ViewContext` object.
- The `HtmlAttributeNotBound` attribute tells MVC that a property isn't set in the HTML.
- You can use dependency injection to inject a custom object into a tag helper.

A tag helper that gets a `Cart` object via dependency injection

```
[HtmlTargetElement("span", Attributes = "my-cart-badge")]
public class CartBadgeTagHelper : TagHelper
{
    private ICart cart;
    public CartBadgeTagHelper(ICart c) => cart = c;

    public bool MyCartBadge { get; set; }

    public override void Process(TagHelperContext context,
        TagHelperOutput output)
    {
        output.Content.SetContent(cart.Count?.ToString());
    }
}
```

The tag helper in a layout

```
<span class="fas fa-shopping-cart"></span>&nbsp;Cart
<span class="badge badge-light" my-cart-badge></span>
```

How to use ViewContext & dependency injection with a tag helper

A tag helper that gets a ViewContext value via dependency injection (part 1)

```
using Microsoft.AspNetCore.Mvc.ViewFeatures;
using Microsoft.AspNetCore.Mvc.Rendering;
...
[HtmlTargetElement("a", Attributes = "[class=nav-link]",
    ParentTag = "li")]
public class ActiveNavbarTagHelper : TagHelper
{
    [ViewContext]
    [HtmlAttributeNotBound]
    public ViewContext ViewCtx { get; set; }

    public override void Process(TagHelperContext context,
        TagHelperOutput output)
    {
        string area =
            ViewCtx.RouteData.Values["area"]?.ToString() ?? "";
        string ctrlr =
            ViewCtx.RouteData.Values["controller"].ToString();
    }
}
```

A tag helper that gets a ViewContext value via dependency injection (part 2)

```
string aspArea = context.AllAttributes["asp-area"]?.Value?
    .ToString() ?? "";
string aspCtrlr =
    context.AllAttributes["asp-controller"].Value
    .ToString();

if (area == aspArea && ctrlr == aspCtrlr)
    output.Attributes.AppendCssClass("active");
}
```

How to create a conditional tag helper

- You can use the `SuppressOutput()` method of the `TagHelperOutput` class to create tag helpers that only send HTML to the browser under certain conditions

A tag helper that only outputs HTML if there's a value in TempData

```
using Microsoft.AspNetCore.Mvc.ViewFeatures;
using Microsoft.AspNetCore.Mvc.Rendering;
...
[HtmlTargetElement("my-temp-message")]
public class TempMessageTagHelper : TagHelper
{
    [ViewContext]
    [HtmlAttributeNotBound]
    public ViewContext ViewCtx { get; set; }

    public override void Process(TagHelperContext context,
        TagHelperOutput output)
    {
        var td = ViewCtx.TempData;
        if (td.Keys.Contains("message"))
        {
            output.BuildTag("h4", "bg-info text-center text-white p-2");
            output.Content.SetContent(td["message"].ToString());
        }
        else
        {
            output.SuppressOutput();
        }
    }
}
```

A layout that uses the conditional tag helper

```
<main>
    <my-temp-message />
    @RenderBody()
</main>
```


Generating URLs in tag helper

- You can use the `GetPathByAction()` method of the `LinkGenerator` class to generate a route-based URL.
- If your tag helper has a constructor with a `LinkGenerator` parameter, MVC will inject it.

Generating URLs in tag helper – Sample Code

A tag helper that generates a paging link (part 1)

```
using Microsoft.AspNetCore.Mvc.ViewFeatures;
using Microsoft.AspNetCore.Mvc.Rendering;
using Microsoft.AspNetCore.Routing;
using Bookstore.Models;
...
[HtmlTargetElement("my-paging-link")]
public class PagingLinkTagHelper : TagHelper
{
    private LinkGenerator linkBuilder;
    public PagingLinkTagHelper(LinkGenerator lg) => linkBuilder = lg;

    [ViewContext]
    [HtmlAttributeNotBound]
    public ViewContext ViewCtx { get; set; }

    public int Number { get; set; }
    public RouteDictionary Current { get; set; }

    public override void Process(TagHelperContext context,
        TagHelperOutput output) {
        // update routes for this paging link
        var routes = Current.Clone();
        routes.PageNumber = Number;
    }
}
```

A tag helper that generates a paging link (part 2)

```
// get controller and action method, create paging link URL
string ctrlr =
    ViewCtx.RouteData.Values["controller"].ToString();
string action =
    ViewCtx.RouteData.Values["action"].ToString();
string url =
    linkBuilder.GetPathByAction(action, ctrlr, routes);

// build up CSS string
string linkClasses = "btn btn-outline-primary";
if (Number == Current.PageNumber)
    linkClasses += " active";

// create link
output.BuildLink(url, linkClasses);
output.Content.SetContent(Number.ToString());
}
```

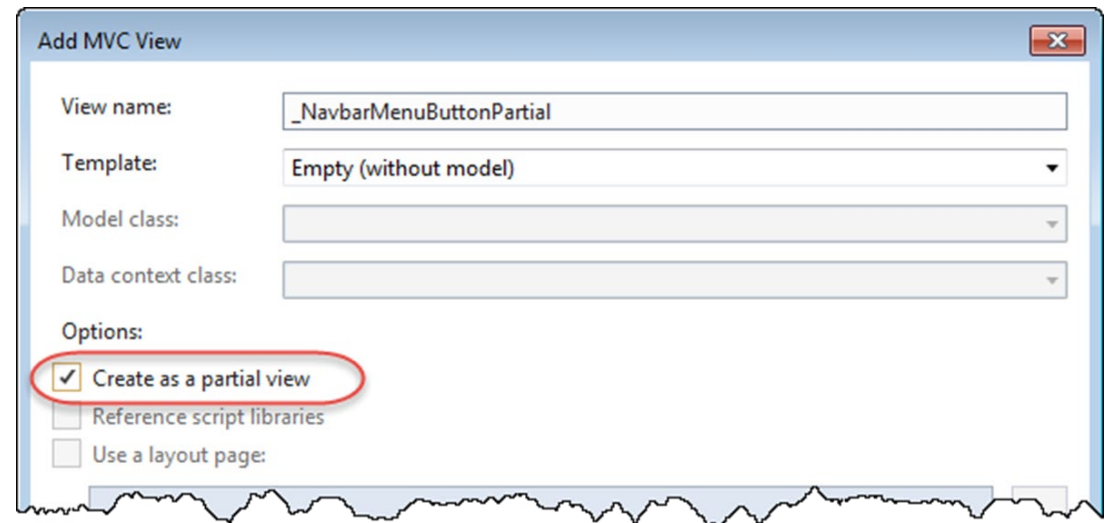
Generating URLs in tag helper – Sample Code

A view that uses the PagingLink tag helper

```
@{
    for (int i = 1; i <= Model.TotalPages; i++)
    {
        <my-paging-link number="@i"
            current="@Model.CurrentRoute" />
    }
}
```

Creating and Using Partial Views

- A **partial view** can contain HTML and Razor code that can be used in multiple view
- How to add a partial view
 - Right click the desired folder location and select Add→View.
 - Use the Razor View template and check the Create as a partial view checkbox.



Creating and Using Partial Views

- A **partial view** can contain HTML and Razor code that can be used in multiple view
- How to add a partial view
 - Right click the desired folder location and select Add→View.
 - Use the Razor View template and check the Create as a partial view checkbox.

A partial view that contains the HTML for a Bootstrap navbar menu button

```
<button class="navbar-toggler" type="button"
    data-toggle="collapse"
    data-target="#menu" aria-controls="menu"
    aria-expanded="false"
    aria-label="Toggle navigation">
    <span class="navbar-toggler-icon"></span>
</button>
```

A layout that uses the partial view in a Bootstrap navbar

```
<nav class="navbar navbar-expand-md navbar-dark bg-dark">
    <partial name="_NavbarMenuButtonPartial" />
    <div class="collapse navbar-collapse" id="menu">
        <ul class="navbar-nav mr-auto">...</ul>
    </div>
</nav>
```

How to pass data to a partial view

- By default, the model object of a partial view is the model object of the parent view.
- By default, the `ViewData` dictionary of a partial view is the `ViewData` dictionary of the parent view.
- If you want the partial view to use a different model or `ViewData` dictionary, you can use the attributes of the partial tag helper to specify that model or dictionary

The `_BookLinkPartial` partial view

```
@model Book
```

```
<a asp-action="Details" asp-controller="Book"  
    asp-route-id="@Model.BookId"  
    asp-route-slug="@Model.Title.Slug()">  
    @Model.Title  
</a>
```

Creating and Using a View Component

- A view component is a class that sends data to a partial view. You can think of a view component as a controller for a partial view.
- To create a view component, you can create a new class file in the Components folder that inherits the `ViewComponent` class. Then, you typically code an `Invoke()` method as described above.
- The partial view for a view component is usually named `Default.cshtml`
- You use tag helper syntax with a prefix of “vc:” to use a view component. For this to work, you must register the custom tag helpers for your app

A view component that passes the Cart count to a partial view

```
using Microsoft.AspNetCore.Mvc;
...
public class CartBadge : ViewComponent
{
    private ICart cart { get; set; }
    public CartBadge(ICart c) => cart = c;

    public IViewComponentResult Invoke() =>
        View(cart.Count);
}
```

The Default.cshtml partial view

```
@model int?
<span class="badge badge-light">@Model</span>
```

The paths that MVC searches for a view component's partial view

```
/Views/ControllerName/Components/ViewComponentName/ViewName
/Views/Shared/Components/ViewComponentName/ViewName
```

How to pass data to a view component

- You can pass data to a view component by coding parameters in the `Invoke()` method. Then, you code the parameter value as an attribute in the tag helper for the view component.
- You can override the default search for the partial view by passing a fully qualified view name and path as the first argument of the `View()` method.

A view component with an `Invoke()` method that has a parameter

```
public class AuthorDropDown : ViewComponent
{
    private IRepository<Author> data { get; set; }
    public AuthorDropDown(IRepository<Author> rep) => data = rep;

    public IViewComponentResult Invoke(string selectedValue)
    {
        var authors = data.List(new QueryOptions<Author> {
            OrderBy = a => a.FirstName
        });

        var vm = new DropDownViewModel {
            SelectedValue = selectedValue,
            DefaultValue = "All",
            Items = authors.ToDictionary(
                a => a.AuthorId.ToString(), a => a.FullName
            );
        };
        return View(
            "~/Views/Shared/Components/Common/DropDown.cshtml", vm);
    }
}
```



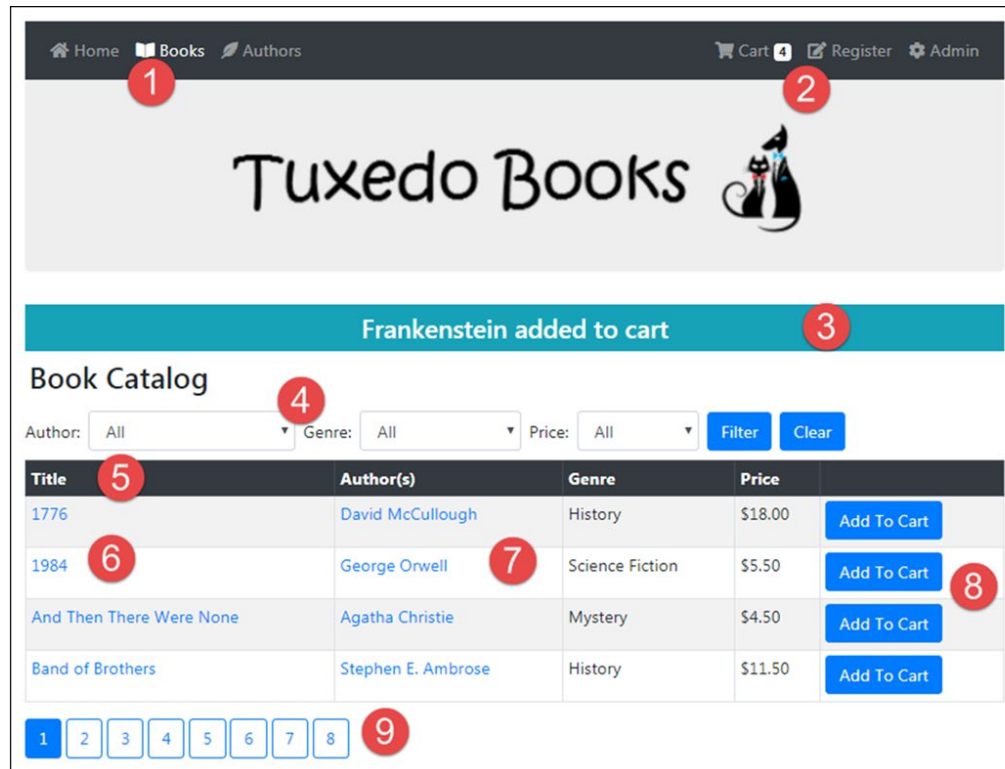

Demo

Code Demos

- **Demo 1 - File:** Ch07aGuitarShop
- **Demo 2 – File:** Ch15Bookstore

Ch15Bookshop Demo Code – tag helpers, partial views and view components in

The Book Catalog page



The custom tag helpers, partial views, and view components in this page

1. The ActiveNavbar tag helper.
2. The CartBadge view component.
3. The TempData tag helper.
4. The AuthorDropDown view component. The other drop-down lists are view components that work similarly.
5. The SortingLink tag helper.
6. The `_BookLink` partial view.
7. The `_AuthorLink` partial view.
8. The Button tag helper.
9. The paging-links partial view, containing the paging-link tag helper.

Summary

- In a view, you can use a Razor **code block** to execute one or more C# statements.
- Within a view, you can code an **inline expression** to evaluate a C# expression and display its result.
- Within a view, you can code **inline loops**. Within these loops, you can use HTML tags to send HTML to the view.
- Within a view, you can code **inline conditional statements** such as if/else and switch statements. Within these statements, you can use HTML tags to send HTML to the view.
- Within a view, you can code an **inline conditional expression** by using C#'s conditional operator (?:) to send HTML tags or plain text to the view. This works like other complex inline expressions that require parentheses.

Summary

- A URL **fragment** allows you to jump to a specified placeholder on a web page. A placeholder is identified by an id attribute, and a fragment is preceded by the hash mark (#) in a URL.
- A **relative URL** is a URL that's relative to the app's root directory.
- An **absolute URL** is a URL that specifies the name of the host.
- A **model** is a regular C# class that defines an object that stores and processes the data for an app.
- Within a view, you can use the Razor `@section` declaration to specify a **section**, which is a block of content that has a name. Then, you can use the `RenderSection()` method to insert the section into a layout.

Summary

- A **partial view** can contain HTML and Razor code that can be used in multiple views within an app or multiple apps.
- A **view component** is a class that sends data to a partial view. You can think of a view component as a controller for a partial view.

Tasks

For this weeks Lab

1. Questions about the assignments

For next weeks Lab

1. Questions about the assignments
2. SCRUM Participation Materials

For your Assignments

1. Working on Routing, controllers and views
2. Testing database connections (where applicable)

Assignments due this week

1. None

Assignments due next week

1. Progress SCRUM 1 & GitCommit – Draft development

Any last minute questions?

INFT3050 – Web Programming

